

tramdag documentation

Contents

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

Install

30 seconds of API

The model in detail: spec → math → networks

Validation

Testing policy

Layout

Citation

Code map — every class and function in `src/tramdag/`

`spec.py` — declare the model

`transforms.py` — the monotone map h and the ordinal transform

`conditioners.py` — the networks behind the terms

`flow.py` — the model

`scores.py` — effect-modifier detection (issue #29)

`utils.py` — helpers that are not about modelling

What is *not* in the package

Where every training hyperparameter lives

Fitting a TRAM-DAG: how training works

How the flow is built: one module, one sub-model per node

How the likelihood is computed

Path A — stochastic optimization (`fit`)

Path B — classical optimization (`fit_classical`)

Memory and disk during fitting

Optimizer choice — current and future

Notation

Symbols

The model's symbols

Plain-text fallback

Per-observation scores & the effect-modifier scan

Worked example: where does $\beta(x)$ need to bend?

How fast can a CausalFlowDAG train?

The options, and how to use them

Method: time-to-target, not loss-go-down

Results

Findings

Recommendation

Varying-coefficient treatment effects: `VC(*modifiers, t=, penalty=)`

Why not `CS("T", "X2", "X3")` ? (anti-pattern)

Semantics

Propensity-centered VC: `center=True` (R-learner orthogonalization)

Validation

tramdag — Interpretable Neural Causal Models (TRAM-DAGs) in PyTorch

Status: beta (0.x), under active development. The API may change between releases until 1.0, so pin a version for reproducibility. Note that this README documents the **unreleased 0.4** API: the term constructors (`SI/CI/VC`), `scores`, `varying_coef` and `intercept_contributions` are not in 0.3.0 on PyPI. Until 0.4 ships, install from git to follow the docs below.

TRAM-DAGs model each variable of a structural causal model with a (transformation-model) flow: one triangular normalizing flow from iid standard-logistic latents to the observed variables. The triangular adjacency structure is exactly your causal DAG. Fit it **once** on observational data and answer all three rungs of Pearl's causal hierarchy — observational (L1), interventional (L2, the do-operator), and counterfactual (L3, Pearl abduction) — while keeping **interpretable effects**: every linear-shift coefficient is a log-odds ratio, exactly as in classical proportional-odds models.

Beate Sick & Oliver Dürr, *Interpretable Neural Causal Models with TRAM-DAGs*, CLeaR 2025 ([arXiv:2503.16206](https://arxiv.org/abs/2503.16206)). This repo is the reference implementation (PyTorch, built on [zuko](#)); all of the paper's experiments are replicated here with pinned tests.

5-minute showcase: the [Colab badge above](#) fits the paper's bimodal benchmark live (GPU-ready) and walks L1 → L2 → L3, every answer checked against analytic ground truth. Further notebooks are available at [notebooks/](#) like the didactic walkthrough of the model: [notebooks/intro_tram_dag.py](#).

Install

```
pip install tramdag # latest release (PyPI)
pip install "git+https://github.com/tensorchiefs/tramdag.git@main" # dev version (track main)
uv sync # or: dev setup from a clone (tests, experiments)
```

Pin the dev install to a commit for reproducibility, e.g. `...tramdag.git@<sha>`.

30 seconds of API

```

from tramdag import CausalFlowDAG, ContinuousNode, OrdinalNode, I, LS, CS

spec = { # the spec IS the labelled DAG
    "X1": ContinuousNode(),
    "X2": ContinuousNode(I("X1")),
    "T": OrdinalNode(2, LS("X1") + CS("X2")),
    "Y": OrdinalNode(4, I("X1") + CS("X2") + LS("T")),
}
flow = CausalFlowDAG(spec) # validates acyclicity, builds the flow

# self-stopping training: per-node plateau lr decay + freezing of converged
# nodes (exact, since the per-node NLLs have independent gradients);
# see docs/training-speed.md for benchmarks and the classic two-phase recipe
flow.fit(
    train_df,
    val_df,
    epochs=4000,
    learning_rate=1e-2,
    batch_size=512,
    schedule="plateau",
    plateau_patience=30,
    freeze_patience=120,
)

# all-`ls` model? fit it classically instead: deterministic float64 L-BFGS,
# exact MLE matching statsmodels/R (see docs/fitting.md)
flow.fit_classical(train_df) # raises on cs/ci specs

flow.log_prob(df) # L1: joint log-likelihood per row
flow.sample(1000) # L1: observational sampling
flow.sample(1000, do={"T": 1}) # L2: interventional (graph mutilation)
flow.pmf(df, node="Y", do={"T": 1}) # L2: analytic interventional PMF

u = flow.abduct(df) # L3 step 1: latents from observations
cf = flow.sample(do={"T": 1}, u=u) # L3 steps 2+3: counterfactuals

flow.ls_coefficients() # interpret: per-edge log-odds-ratios (LS terms)
flow.intercept_contributions("Y", df) # interpret: per-parent partial effects
# of an additive complex intercept (centered)

# heterogeneous treatment effects: a small, penalized effect head beta(x)*T
# (VC term) with a first-class read-out – see docs/varying-coefficients.md
# e.g. CS("X1", "X2") + VC("X1", t="T") ->
# flow.varying_coef("Y", df) # beta(x): deterministic, y-free

flow.scores(df, node="Y") # per-observation scores dI/dtheta
flow.effect_modifier_scan(df, "Y", t="T") # which VC modifiers? (CUSUM)
# scan from a cheap all-ls fit – docs/scores.md

flow.save("flow.pt")
flow = CausalFlowDAG.load("flow.pt")

```

The model in detail: spec → math → networks

Per node, the transformation is additive on the latent (log-odds) scale — one intercept term I plus any number of shifts (notation: [docs/notation.md](#)):

$$u = h_{\theta}(x) + \sum \beta \cdot x_{pa} + \sum g(x_{pa}) + (\beta_{\theta} + b_{\theta}(x_{mod})) \cdot x_t$$

term	math	what gets built	interpretability
			the baseline transform

term	math	what gets built	interpretability
<code>I()</code> / bare <code>I</code> / omitted	$h_{\theta}(x) \text{ — constant } \theta$	SimpleIntercept: one free parameter vector, no network	
<code>I("A")</code>	$h_{\theta}(a)(x) \text{ — } \theta$ bends with the parent	ComplexIntercept: $\text{MLP } [8, 8] \rightarrow n_{\text{params}}$	the parent reshapes the whole distribution; no single coefficient
<code>I("A", "B")</code> (default <code>allow_interaction=True</code>)	$h_{\theta}(a, b)(x)$	one joint MLP over both parents — they interact in θ	maximal flexibility
<code>I("A", "B", allow_interaction=False)</code>	$h_{\theta}(a) + \theta(b)(x)$	one MLP per parent , parameter vectors summed in coefficient space	per-parent partial effects via <code>flow.intercept_contributions</code>
<code>LS("A")</code>	$\beta \cdot a$	<code>Linear(width, 1)</code> , no bias — one parameter	$\exp(\beta)$ is an odds ratio
<code>CS("A")</code>	$g(a)$, additive	ComplexShift: $\text{MLP } [64, 128, 64] \rightarrow 1$	plot g
<code>CS("A", "B")</code>	$g(a, b)$ — joint	one MLP over the concatenated features	interaction <i>in the shift</i>
<code>CS("A") + CS("B")</code>	$g_1(a) + g_2(b)$	two MLPs, scalars added	GAM-style, each effect plottable
<code>VC("A", "B", t="T")</code>	$(\beta_0 + b_{\theta}(a, b)) \cdot x_t$	scalar β_0 + zero-initialised penalized MLP $[16] \rightarrow 1$; the treatment value multiplies, it never enters the net	$\beta_0 \approx$ constant effect, <code>flow.varying_coef</code> reads $\beta(x)$

A node takes **at most one I term with parents** — a term list is therefore always purely additive on the latent scale, and interactions exist only *inside* a term. Lists and `+` sums are interchangeable: `[LS("A"), CS("B")]  $\equiv$  LS("A") + CS("B")`.

Two knobs on the terms:

- **transform= on I** picks the basis of h_{θ} for a continuous node — "bernstein" (default, 20 coefficients, tails extrapolate with the boundary slope), "spline" (monotone RQ spline, 23 params at `bins=8`, fixed tail slope) or "affine" (2 params: the latent is exactly logistic). Ordinal nodes have no basis: their intercept is the cutpoint vector, $P(x \leq k) = \sigma(\theta_k - \text{shift})$.
- **units= on I/CS/VC** sizes the term's network, e.g. `units=[16]` for one hidden layer of 16 neurons. The defaults match the PyTorch reference this package grew out of ([buehlpa/TramDag](#), `tram_models.py`), **not** the TRAM-DAG paper's own R nets — so a replication sets `units=` and `activation=` explicitly, as the configs in `experiments/paper/` do.

Feature widths: a continuous parent enters raw (1 column), an ordinal parent one-hot (`levels` columns). Abduction is exact for continuous nodes and truncated-logistic for ordinal ones, so `flow.sample(u=flow.abduct(df))` reproduces `df` exactly / level-exactly.

There are two ways to fit the model: a stochastic deep learning optimizer (`fit`) and a 2nd order optimization like in the classical statistical models (`fit_classical`). The latter is more efficient for all-`ls` models (each node-conditional is then a classical transformation model). For more details, see the [docs/fitting.md](#) file.

Validation

Two layers, deliberately separate.

The framework's own test suite ([tests/](#)) measures the library against three inline data-generating processes it carries itself, so it needs no research code to run. Its strongest claim is an equality, not a similarity: an all-`ls` outcome node *is* an ordered-logit model, so the flow's MLE must match `statsmodels.OrderedModel` on the same design matrix — it does, and the varying-coefficient acceptance bars (effect recovery, propensity centering) are pinned the same way. What the tests guarantee, and how each ground truth is obtained, is documented in [tests/README.md](#).

The paper replications ([experiments/](#)) are separate: one self-contained script per dataset, with its hyperparameters in a sibling YAML file and its expected results committed under each area's `ground_truth/`. A dedicated workflow runs them and compares.

experiment	paper	demonstrates
<code>triangle.py</code> (<code>linear-ls</code> , <code>linear-cs</code> , <code>atan-cs</code> , <code>sin-cs</code>)	§6.1, C.3	LS coefficient recovery ($\beta = 2, -0.2, +0.3$), CS curve $\equiv -f(x_2)$ for non-monotone f
<code>triangle_mixed.py</code> (<code>linear-ls</code> , <code>exp-cs</code>)	§6.2	mixed data L1/L2 + the C.4 odds-ratio check ($OR \approx 7.4$)
<code>vaca.py</code>	§5.1–5.2	the bimodal L1 case a default CNF misses; L2 $p(x_3 \mid do(x_2))$ against analytic means
<code>carefl.py</code>	§5.3	L3 counterfactual curves vs analytic truth
<code>validate_ls.py</code>	—	flow $\equiv$ <code>statsmodels</code> $\equiv$ R <code>MASS::polr</code> to ~4 decimals, on a frozen synthetic cohort with a known true effect

Sign note: ordinal shifts are *subtracted* here but *added* in the paper, so fitted ordinal weights are the paper's with flipped sign (each `truth.json` records both conventions). A figure-by-figure account of what is reproduced, and what is not (the competing CNF/NSF baselines), is in [experiments/paper/PAPER_COVERAGE.md](#).

Training speed — schedules, per-node freezing, L-BFGS and device benchmarks: [docs/training-speed.md](#).

Testing policy

See the [tests/README.md](#) file for more details.

Layout

<code>src/tramdag/</code>	<code>spec.py</code> <code>transforms.py</code> <code>conditioners.py</code> <code>flow.py</code>
<code>tests/</code>	<code>scores.py</code> <code>utils.py</code> <- the framework, and nothing else
<code>experiments/</code>	unit tests, identities, acceptance bars, three inline DGPs
	research code, one directory per area, each self-contained:
	<code>paper/</code> the replications + generators + frozen data
	<code>benchmarks/</code> training-speed and machine measurements
	<code>misc/</code> the classical-MLE validation
	<code>paper/misc:</code> <code><name>.py</code> + <code><name>.yaml</code> , <code>data/</code> ,
	<code>ground_truth/</code> , <code>tests/</code> , <code>results/</code> (<code>benchmarks</code> reads the
	other areas' data and pins no ground truth)
<code>notebooks/</code>	four executed examples: didactic intro, Colab demo,
	additive-vs-joint intercepts, varying coefficients
<code>docs/</code>	<code>code-map.md</code> (every class/function + all knobs),
	<code>fitting.md</code> , <code>notation.md</code> , <code>training-speed.md</code> ,
	<code>varying-coefficients.md</code> , <code>scores.md</code>

Implementation conventions (latent-scale signs, raw/one-hot parent encoding, log-space ordinal likelihood, seeding) are documented in [CLAUDE.md](#) and pinned by tests.

Citation

If you use `tramdag`, please cite the method paper:

```
@inproceedings{sick2025tramdag,  
  title      = {Interpretable Neural Causal Models with TRAM-DAGs},  
  author     = {Sick, Beate and D{\u}rr, Oliver},  
  booktitle  = {Proceedings of the 4th Conference on Causal Learning and Reasoning (CLear)},  
  series     = {Proceedings of Machine Learning Research},  
  volume     = {275},  
  year      = {2025},  
}
```

Code map — every class and function in `src/tramdag/`

One entry per name, with its role and its place in the pipeline. Names in parentheses are private machinery: useful to know, not part of the API. The last section lists every training hyperparameter and where it lives.

`spec.py` — declare the model

Every term constructor has a pythonic name and a short alias; the two are the same object, so `LS` is `linear_shift`.

Name	Role
<code>Term</code>	One additive term of a node's transformation: the frozen triple <code>(effect, parents, options)</code> . <code>+</code> on terms builds plain lists. Effect-specific settings live in <code>options</code> and read as attributes (<code>term.penalty</code> , <code>term.units</code> , ...).
<code>simple_intercept()</code> / <code>SI</code>	The parentless intercept — the paper's SI. Free transform parameters, the same for every row. Carries the basis choice (<code>transform=</code> , default "bernstein"); extra keyword arguments pass straight to the transform class.
<code>complex_intercept()</code> / <code>CI</code>	The parent-conditioned intercept — the paper's CI: the parents reshape the monotone transform. Needs at least one parent. Also carries <code>units=</code> and <code>allow_interaction=</code> (joint vs. additive multi-parent intercept).
<code>intercept()</code> / <code>I</code>	The fallback: dispatches on its arguments to <code>SI</code> (no parents) or <code>CI</code> (parents). The bare names <code>I</code> and <code>SI</code> in a term list both mean the simple intercept.
<code>linear_shift()</code> / <code>LS</code>	Linear shift $\text{beta} * x$ — the interpretable log-odds coefficient. Exactly one parent.
<code>complex_shift()</code> / <code>CS</code>	Complex shift: an MLP $g(x)$, additive on the latent scale. Several parents form one joint network.
<code>varying_coefficient()</code> / <code>VC</code>	Varying-coefficient shift $(\text{beta0} + \text{b_theta(mods)}) * x_t$ — the penalized treatment-effect head (issue #28). <code>center=</code> adds propensity centering (issue #30).
<code>ContinuousNode</code>	Continuous variable: monotone 1-D transform plus shifts. <code>terms</code> is the first positional argument.
<code>OrdinalNode</code>	Ordinal variable with <code>levels</code> classes: ordered logit (cutpoints) plus shifts.
<code>node_terms()</code> / <code>node_parents()</code>	Canonical term list / ordered de-duplicated parent names of a node.
<code>validate_and_sort()</code>	Edge-ownership validation plus Kahn topological sort. The returned order makes the flow triangular.
<code>spec_to_dict()</code> / <code>spec_from_dict()</code>	Checkpoint (de)serialization. A term serializes as <code>{effect, parents, options}</code> and nothing else, since <code>options</code> is already canonical. No compatibility shims, and <code>spec_from_dict</code> builds <code>Term</code> directly — so <code>validate_and_sort</code> is the only guard on that path.
<code>(_normalize_terms, _as_term,</code> <code>_intercept_basis, _options,</code> <code>_OPTION_DEFAULTS)</code>	Formula flattening and per-entry validation (a <code>+</code> sum nested in a list is rejected), the one-parented- <code>I</code> rule plus basis hoisting in one pass, canonical option storage.

Name	Role
(<code>_check_term</code> , <code>_check_node</code> , <code>_check_vc_term</code> , <code>_kahn_sort</code>)	The stages behind <code>validate_and_sort</code> : per-term validation (effect, LS arity, unknown parents), per-node edge-ownership bookkeeping, the VC-specific checks, and the topological sort.

transforms.py — the monotone map h and the ordinal transform

Name	Role
<code>StandardLogistic</code>	The TRAM base distribution: <code>log_prob</code> , <code>sample</code> (generator-aware), <code>icdf</code> .
<code>BernsteinUT</code>	Bernstein-polynomial transform (default basis, <code>n_coeffs=20</code>). Linear tail extrapolation follows the boundary derivative. <code>marginal_init_theta()</code> gives the calibrated start used by <code>fit(marginal_init=True)</code> .
<code>SplineUT</code>	Monotone rational-quadratic spline (<code>bins=8</code>). Tails extrapolate with a <i>fixed</i> slope — the structural reason spline trails Bernstein on tail-heavy data.
<code>AffineUT</code>	Monotone affine transform: the node-conditional is a logistic GLM.
<code>make_univariate_transform()</code>	Basis registry: name $\rightarrow$ transform instance.
<code>ordinal_cutpoints()</code>	Unconstrained $(n, K-1) \rightarrow$ increasing cutpoints with $\pm\infty$ ends. Port of the original parametrization.
<code>ordinal_log_prob()</code>	$\log P(Y=y)$, computed in log-space. Load-bearing: the naive sigmoid difference saturates in float32 and freezes nodes at init. Do not simplify.
<code>ordinal_pmf()</code> / <code>ordinal_sample()</code> / <code>ordinal_abduct()</code>	Class probabilities / latent $\rightarrow$ level / truncated-logistic latent recovery (Pearl step 1) for ordinal nodes.
<code>ordinal_marginal_init_theta()</code>	Cutpoint start that matches the empirical class frequencies (<code>marginal_init</code>).
(<code>_ScaledUT</code> , <code>_expanding_bisection</code> , <code>_bounds</code> , <code>_log1mexp</code>)	Quantile pre-scaling base class; tail-safe inverse; per-level cutpoint intervals; stable $\log(1-\exp(x))$.

conditioners.py — the networks behind the terms

Default architectures replicate the PyTorch reference this package grew out of ([buehlpa/TramDag](#), `tram_models.py`), so a fitted model stays comparable to it. They are **not** the TRAM-DAG paper's nets: the paper's R code uses `c(2, 25, 25, 2)` with sigmoid for the triangle experiments and a 10-100 tanh net for its CAREFL/VACA comparisons, so each config in `experiments/paper/` states `units=` and `activation=` itself.

Name	Term	Role
<code>SimpleIntercept</code>	bare <code>I</code>	Free parameter vector; no parents.
<code>ComplexIntercept</code>	<code>I(...)</code>	8-8 ReLU MLP from parent features to the transform parameters.
<code>LinearShift</code>	LS	<code>Linear(n, 1, bias=False)</code> . <code>.weight</code> is the interpretable coefficient; no bias because the intercept slot owns the constant.
	CS	64-128-64 ReLU MLP to one shift value.

Name	Term	Role
ComplexShift		
VaryingCoefficient	VC	$\text{beta0} + \text{b_theta}(\text{mods})$ with a zero-initialized output layer and the L2 hook <code>l2()</code> . <code>beta()</code> evaluates the effect, <code>recenter()</code> re-splits $\text{beta0}/\text{b_theta}$ after training (function-preserving).
(<code>_mlp</code>)	—	The one MLP builder: a ReLU stack of the given <code>units</code> , then a bias-free output layer.

flow.py — the model

Name	Role
<code>CausalFlowDAG</code>	The flow: one <code>_Node</code> per variable in topological order. Construction seeds the weights (<code>seed=</code> is the reproducibility knob).
<code>fit()</code>	Joint maximum likelihood with Adam, one parameter group per node (exact, because the NLL decomposes per node). Options: <code>schedule="plateau"</code> (per-node decay), per-node freezing, <code>restore_best</code> , <code>marginal_init</code> , <code>vc_warm_start</code> , <code>plateau_factor</code> , <code>vc_oof_fit</code> . A second call continues training. Progress goes to the <code>tramdag.flow</code> logger.
<code>fit_classical()</code>	Float64 full-batch L-BFGS for all- <code>ls</code> specs: deterministic, exact MLE, matches <code>statsmodels/R polr</code> . Refuses flexible specs.
<code>sample()</code>	Observational, interventional (<code>do=</code> , graph mutilation) and counterfactual (<code>u=</code>) sampling.
<code>abduct()</code>	Pearl step 1: recover the latents. Continuous exactly, ordinal by truncated draw.
<code>pmf()</code>	Analytic class probabilities of an ordinal node, with <code>do=</code> overrides.
<code>log_prob()</code> / <code>nll()</code>	Joint per-row log-likelihood / mean per-node NLL diagnostic.
<code>node_log_prob()</code>	The per-node decomposition everything trains and evaluates through.
<code>varying_coef()</code>	Closed-form read-out $\text{beta}(x)$ of a fitted VC term. Deterministic, y-free.
<code>scores()</code> / <code>effect_modifier_scan()</code>	Analytic per-observation scores and the CUSUM modifier scan (delegate to <code>scores.py</code>).
<code>intercept_contributions()</code>	Post-hoc GAM-style decomposition of a complex intercept into mean-centered per-term parts.
<code>ls_coefficients()</code>	The per-node linear-shift weights — the interpretable coefficients.
<code>design_matrix()</code>	Parent encoding as a DataFrame (<code>drop_first=</code> gives the classical <code>statsmodels/polr</code> design).
<code>to_matrix()</code>	The labeled meta-adjacency matrix of term effects.
<code>save()</code> / <code>load()</code>	Checkpoints with history and machine provenance. <code>load</code> requires a complete checkpoint and fails loudly otherwise.
(<code>_Node</code> , <code>_VCGroup</code>)	Per-node module (intercept + shift <code>ModuleDict</code> + VC bookkeeping); construction is <code>_build_intercept/_build_shifts</code> , and <code>theta_shift()</code> computes $(\text{theta}, \text{shift})$ through <code>_theta/_vc_shift/vc_column</code> .

Name	Role
(<code>_FitSchedule</code> , <code>_fit_epoch</code> , <code>_end_epoch</code> , <code>_make_optimizer</code> , <code>_val_nll</code> , <code>_vc_penalized</code> , <code>_log_epoch</code> , <code>_snapshot_best</code> , <code>_load_best_weights</code> , <code>_best_store</code>)	The fit loop, decomposed: per-node plateau/freeze bookkeeping, one minibatch epoch (with the frozen carry-forward), the per-epoch record/schedule/snapshot/log step, and the restore-best store that persists across <code>fit</code> calls.
(<code>_node</code> , <code>_encode_parent</code> , <code>_features</code> , <code>_tensorize</code> , <code>_generator</code> , <code>_dtype</code> , <code>_np_dtype</code> , <code>_feat_width</code> , <code>_slice_ehat</code> , <code>_term_cells</code>)	Node lookup with one shared error; parent encoding (continuous raw, ordinal one-hot); <code>_tensorize(df, cols=None)</code> for any column subset; seeded-generator, dtype, feature-width and adjacency-cell plumbing.
(<code>_set_ranges</code>)	Train 5%/95% quantiles onto the transform domain, plus the optional marginal initialization. First fit only.
(<code>_vc_warm_start</code> , <code>_ls_proxy_spec</code> , <code>_vc_oof_stage</code> , <code>_vc_oof_propensity</code> , <code>_predict_p1</code> , <code>_binary_p1</code> , <code>_vc_ehat_live</code> , <code>_vc_ehat_columns</code> , <code>_recenter_vc</code> , <code>_source_proxies</code>)	The VC machinery: classical β_0 warm start; frozen out-of-fold propensities for training (DML); live full-fit propensities for inference (recomputed under <code>do</code> , never cached); post-fit re-centering.
(<code>_is_all_ls</code> , <code>_covered_by_classical</code>)	Guard for <code>fit_classical</code> : every term an LS, or a parentless <code>I()</code> basis carrier.

scores.py — effect-modifier detection (issue #29)

Name	Role
<code>node_scores()</code>	Analytic, exact per-observation scores $\psi_i = d l_i / d \theta$ for every LS weight and VC β_0 . No autograd.
<code>effect_modifier_scan()</code>	Zeileis-Hornik fluctuation scan: order the treatment scores by each candidate, $\sup$
<code>sup_bb_pvalue()</code>	$P(\sup)$
(<code>_dl_ds</code> , <code>_ls_score_columns</code> , <code>CRIT_5PCT</code>)	Closed-form latent-scale derivative; the LS/one-hot score-column builder; the 5% critical value 1.3581.

utils.py — helpers that are not about modelling

Nothing is imported at module level here: `config_section` needs no dependency, and `machine_info` pulls in torch and platform only when called.

Name	Role
<code>config_section()</code>	Pick a mapping out of an already-parsed configuration (descending through any number of keys) and require an exact key set, so a missing key cannot become a hidden default and an extra one cannot look effective. Parsing stays with the caller, so the package depends on no config parser.
<code>machine_info()</code>	Machine/software snapshot stored by <code>save()</code> , so timings stay comparable across machines. Never raises. Exported top-level as <code>tramdag.machine_info</code> .

What is *not* in the package

The SCM generators, the frozen datasets and the replication scripts are research code and live in [experiments/](#), outside the installed package — see [experiments/README.md](#). The framework's own tests do not depend on them: they measure against three inline DGPs in [tests/conftest.py](#).

Where every training hyperparameter lives

Everything that shapes a fit is either a keyword you pass or a documented default you can read at the call site. Nothing numeric is buried.

Knob	Where	Default
learning rate, batch size	<code>fit()</code>	1e-2 / 512 (in-repo callers state them explicitly anyway)
schedule, plateau patience, plateau decay factor	<code>fit(schedule=, plateau_patience=, plateau_factor=)</code>	None / 30 / 0.3 (lr floor $1e-3 * \text{learning_rate}$, stated in the docstring)
per-node freezing	<code>fit(freeze_patience=, min_delta=)</code>	off / 1e-4 (freeze guard $1e-2 * \text{learning_rate}$)
early stopping	<code>fit(restore_best=)</code>	False = exact MLE
calibrated init	<code>fit(marginal_init=)</code>	False (pure init, MLE unchanged)
VC warm start	<code>fit(vc_warm_start=)</code>	True (classical β_0 start)
VC stage-1 proxy fits	<code>fit(vc_oof_fit=)</code>	<code>{"epochs": 300, "learning_rate": 1e-2, "batch_size": 512}</code>
VC penalty and centering	<code>VC(penalty=, center=, center_folds=)</code>	1.0 / False / 5
L-BFGS budget	<code>fit_classical(max_iter=, tol=, chunk=, history_size=)</code>	400 / 1e-6 / 25 / 50
training budget	<code>fit(epochs=)</code>	required — a fixed default is wrong in both directions (training-speed)
network widths	<code>units=</code> on I/CS/VC	(8, 8) / (64, 128, 64) — parity with the PyTorch reference's default classes; VC's (16,) has no counterpart there and comes from the recovery measurement
activation	<code>activation=</code> on I/CS/VC	"relu" (the reference default classes); "sigmoid" and "tanh" are the paper's
transform basis	<code>I(transform=, **kwargs)</code> (extra kwargs go to the transform class)	"bernstein", <code>n_coeffs=20</code> unconstrained coefficients (zuko ties two more control points on, so order 21); spline <code>bins=8</code> = zuko's NSF default (the domain is fixed at [-5, 5], <code>transforms.BOUND</code>)
shuffling / weight init	<code>fit(seed=)</code> / <code>CausalFlowDAG(seed=)</code>	init happens at construction — the constructor seed is the reproducibility knob

Fitting a TRAM-DAG: how training works

This page is the technical reference for `CausalFlowDAG`. It explains how the module builds the flow and how it computes the likelihood. It describes the two fitting paths: the stochastic optimizer (`fit`) and the classical optimizer (`fit_classical`). It also covers per-node freezing, the memory model, and future directions for optimizer choice.

How the flow is built: one module, one sub-model per node

A `CausalFlowDAG` is a single `torch.nn.Module`, but it is **not one monolithic network**. At construction (`CausalFlowDAG.__init__`), it topologically sorts the DAG. It then builds an `nn.ModuleDict` with **one `_Node` sub-module per variable**. The nodes share *no parameters*. Each node owns the pieces for its own conditional $p(x_i | pa(x_i))$:

- an **intercept** that produces the transform parameters θ . This is `SimpleIntercept`, a free parameter vector that is data-independent. If the node has `ci` parents, it is `ComplexIntercept` instead, a small MLP whose output θ depends on those parents.
- a **monotone 1-D transform** h (`BernsteinUT` / `SplineUT` / `AffineUT`). The transform itself carries **no learnable weights**, only the fitted range buffers `xmin/xmax`. The θ from the intercept sets its shape entirely.
- a `ModuleDict` of **shift modules**, one per shift *term* — which is one per parent edge except for a joint `CS("a", "b")`, a single module keyed `"a+b"` that owns both edges: `LinearShift` (LS, a single weight), `ComplexShift` (CS, an MLP) or `VaryingCoef` (VC, β_0 plus a penalized head).

So is this one network or several? It is **one module, several independent per-node sub-models** (each itself a small intercept + shifts assembly). One module bundles them, and one optimizer trains them, but their parameters are disjoint. The DAG structure lives entirely in *which parents each node reads*. There is no edge weight matrix or shared trunk.

For a batch, `_Node.theta_shift` assembles a node's θ (from the intercept) and total `shift` (sum of its shift modules over the parent features). `_encode_parent` encodes the parent features: continuous parents raw, ordinal parents one-hot.

How the likelihood is computed

A TRAM-DAG maps iid standard-logistic latents u to the observed x in causal order. Node i reads only its parents (earlier variables, as *data*). Therefore the Jacobian of $u \rightarrow x$ is **triangular**, and its log-determinant is the sum of the per-node 1-D terms. The joint log-likelihood therefore **decomposes per node**:

$$\log p(x) = \sum_i \log p(x_i | pa(x_i))$$

`CausalFlowDAG.node_log_prob` computes one term per node and `log_prob` sums them. For a node, given θ , `shift` from `theta_shift`:

- **continuous** — change of variables through the monotone transform:

$$\begin{aligned} u &= h(x; \theta) + \text{shift} \\ \log p(x | pa) &= \log f_{\text{logistic}}(z) + \log |dz/dx| \end{aligned}$$

That is, the term is the standard-logistic density at the latent u (`StandardLogistic.log_prob`) plus the transform's log-derivative. `ut.forward` returns this log-derivative as `ladj`. This is the 1-D Jacobian term that makes the result a proper density, not only a score.

- **ordinal** — an ordered-logit / proportional-odds head, $P(x \leq k) = \sigma(\theta_k - \text{shift})$. `ordinal_log_prob` evaluates it as the log of the cutpoint-interval probability. The computation runs in log-space via `logsigmoid/loglmexp`, because the naive sigmoid difference underflows to exactly-zero gradients in `float32`.

The training loss is the summed per-node **mean** NLL over the batch ($\sum_i \text{mean_rows}(-\log p(x_i | \text{pa}))$). In contrast, `log_prob` returns the per-row joint, which scores whole observations.

Consequence used by both optimizers: because parents enter as data, the per-node gradients are independent. Therefore a joint fit of the summed loss is identical to a separate fit of each node. This independence licenses per-node learning rates, per-node freezing (below), and the all-`ls` classical fit.

Path A — stochastic optimization (`fit`)

`CausalFlowDAG.fit` is the general-purpose trainer. Any `cs/ci` edge requires it. Mechanics:

- **Adam with one parameter group per node.** `fit` builds each group from `self.nodes[name].parameters()`, so each node can carry its own learning rate.
- **Minibatches:** a fresh `torch.randperm` shuffle occurs each epoch. The loss is the summed per-node mean NLL on the batch.
- **Transform ranges:** `_set_ranges` sets them once at the start from the train 5%/95% quantiles. These ranges define each Bernstein/spline domain.
- **Learning-rate schedules** (`schedule=`): `None` (constant) or `"plateau"`, which decays each node's `lr` off *its own* validation NLL. (`"onecycle"` and `"cosine"` lost to plateau in the June 2026 benchmark and were removed in 0.4.0.)
- **Per-node freezing** (`freeze_patience=`, see below).
- **restore_best:** when enabled, `fit` snapshots each node's best-validation weights and restores them at the end (early-stopping regularization). The default is `False`, so the fit sits at the training-data MLE.
- **marginal_init=:** opt-in calibrated initialization of the unconditional intercepts — Bernstein nodes start at the linear map onto the latent 5%/95% quantiles, ordinal cutpoints at the empirical class log-odds. A pure init; the converged MLE is unchanged.
- **vc_warm_start=** (default on): each `vc` term's `beta0` starts at the classical all-`ls` solution of its node's conditional, so the penalized head only learns deviations.
- **plateau_factor=** (default 0.3): the multiplier of one per-node plateau decay step. Read only under `schedule="plateau"`; the floor is `1e-3 × learning_rate`, and under this schedule a node may only freeze once its `lr` has decayed to `1e-2 × learning_rate`.
- **vc_oof_fit=:** keyword overrides for the stage-1 out-of-fold propensity fits behind `VC(center=True)`, merged over `{"epochs": 300, "learning_rate": 1e-2, "batch_size": 512}`. Ignored when the treatment node is all-`ls`, which takes the deterministic `fit_classical` path instead.

How per-node freezing works

Set `freeze_patience=N` to remove converged nodes from training. Each epoch, `fit` tracks every node's own validation NLL and counts epochs since its last improvement (by more than `min_delta`). When that counter reaches `N`, `fit` **freezes** the node. Under `schedule="plateau"`, the freeze occurs only after the schedule decays the node's learning rate. This order prevents a freeze while a smaller step can still help.

A freeze does more than stop updates. `fit` **excludes the frozen node from the computation entirely**: it calls `node_log_prob(values, nodes=active)` with only the still-active nodes, so a frozen node runs no forward and no backward pass. That is a real FLOP saving. It is also *correct*, precisely because the per-node gradients are independent (above): removal of a converged node's term from the loss does not change any other node's gradient. The slowest node sets the total training time. Once the other nodes converge, they cost nothing.

When **every** node is frozen, the fit returns early (the self-stopping behavior). `fit` records the freeze epochs in `flow.history["frozen"]`. The freeze state is local to a single `fit` call (a later `fit` call trains all nodes again). This composes with `restore_best`, which still restores each node's best-validation snapshot at the end.

Freezing vs. parallelism (why one helps and the other usually does not)

A tempting argument says: "freezing a node saves time, so node computation costs time, so parallel computation of nodes will also save time." The premise is correct, but the conclusion does not follow. The two act through different mechanisms:

- **Freezing removes work.** The frozen node's forward/backward never runs again. Removal of work is an *unconditional* win, and most of its payoff is structural. Once *all* nodes freeze, the fit stops and deletes whole epochs. Parallelism can never do that (it can shrink time-per-epoch, not the number of epochs).
- **Parallelism only overlaps work.** It runs the same computations concurrently, and this helps *only if the hardware sits idle* during the sequential node loop. The hardware usually is not idle. Parallelism already exists one level down: each node's batched tensor ops run across all rows, and the BLAS/GPU backend already uses all cores. While node A's matmul runs, the cores are busy, so node B "at the same time" only time-slices the same cores. The result is contention, often slightly slower.

So the speed gain from freezing does **not** imply spare capacity for parallel work across nodes. The exception is the regime where per-node ops *under-utilize* the hardware: small batches, tiny models, or many small nodes on a big GPU where each kernel leaves it mostly idle. In that regime, node overlap can help. But the right tool is **fusion** (batch same-shaped nodes into one larger tensor op, or CUDA streams), not a threaded Python loop (which the GIL fights). That is the "vectorize/fuse the per-node loop" direction in [Optimizer choice](#) below. It is a conditional win in that regime, distinct from the unconditional win of freezing.

Benchmarks, schedule trade-offs, and the recommended self-stopping recipe are in [training-speed.md](#). The worked walkthrough is [notebooks/intro_tram_dag.py](#).

Path B — classical optimization (`fit_classical`)

`CausalFlowDAG.fit_classical` is the dedicated optimizer for **all-1s** models, where every node-conditional is a classical transformation model (ordered-logit / Colr). It raises on any `cs/ci` edge.

- **Full-batch, float64, L-BFGS** (strong-Wolfe line search). There are no minibatches, no schedule, and no early stopping. Therefore the fit is **deterministic** (same init → bit-identical) and lands on the **exact MLE** (matches `statsmodels/R` to $\sim 1e-3$ on well-identified coefficients).
- **Solver budget** (`max_iter=400`, `tol=1e-6`, `chunk=25`, `history_size=50`): the fit runs in rounds of `chunk` inner L-BFGS iterations and checks NLL flatness between them, so `n_iter` in the report counts in multiples of `chunk`. `history_size` is the L-BFGS memory.
- **float64 is a transient compute mode**: `self.double()` upcasts parameters *and* the transforms' range buffers in one call. The fit runs in double. A *finally*: `self.float()` restores float32. The

stored model stays float32, and so do `save/load`. The data path (`_tensorize`, `sample`, `pmf`) reads the model dtype, so it follows along.

- **Convergence:** the flag comes from NLL flatness and is *advisory*. A Bernstein intercept and weakly-identified directions (rare one-hot levels, a flat treatment-effect ridge) continue to drift along zero-curvature valleys after the likelihood is at the optimum. Correctness comes from a comparison with classical software (`python -m misc.validate_ls classical`), not from the flag.
- Read the fitted coefficients with `ls_coefficients()`.

Warm-start handoff: classical fit, then keep training

`fit_classical` leaves the model at the MLE in float32, ready for any normal operation — and continuing with `fit()` from there **stays put**, which is both a check that the classical solution really is the optimum and a way to use it as a fast, principled initialization:

```
flow.fit_classical(train_df)                # exact MLE, seconds
before = flow.ls_coefficients()["y"]
flow.fit(train_df, epochs=300, learning_rate=1e-3, # a gentle Adam phase ...
        restore_best=False, verbose=0)
after = flow.ls_coefficients()["y"]          # ... barely moves
```

A small drift means the classical fit was already at the optimum. The same handoff is what `fit(vc_warm_start=True)` uses internally to start a varying-coefficient term from the classical linear-shift coefficient.

Memory and disk during fitting

Neither fitting path writes to disk. Everything during `fit/fit_classical` lives in RAM:

- model parameters and (for `fit`) the per-node Adam optimizer state
- the `history` dict (per-epoch train/val NLL, lr, wall-clock), which is an in-memory attribute and not a file
- `restore_best` snapshots: `copy.deepcopy` of node `state_dicts` held in memory (`flow.py`), never serialized

The **only** disk I/O in the module is the explicit, user-called `save` (`torch.save` of `spec` + `state_dict` + `history`) and its counterpart `load`. So a fit produces no temp files, no checkpoints, and no scratch directory.

Persistence is opt-in via `flow.save(path)`. (The *experiment scripts* write the `results/` and `docs/perf/` artifacts in this repo, not the library's fitting code.)

Optimizer choice — current and future

Today: **Adam** for flexible models, **L-BFGS** (float64) for all-`ls`. The per-node-param-group structure already in `fit` makes several extensions cheap. These extensions are worth consideration as the package matures:

- **IRLS / Fisher scoring for the all-`ls` path.** The classical fitting algorithm for proportional-odds / GLM-type models is iteratively reweighted least squares (Newton with the expected information). For the `ls` case, it will likely reach the MLE in *fewer, more stable* steps than L-BFGS. Also, the Fisher information that it computes is exactly the ingredient for the planned **standard-error table** (currently flagged as next in the CHANGELOG). It is a strong candidate to back `fit_classical` in a future version.

- **Second-order / curvature-aware methods for flexible nodes.** K-FAC or a Gauss-Newton approximation can help the `ci/cs` MLPs. However, full-batch quasi-Newton is a poor fit for them (minibatch noise also regularizes, which is why `fit_classical` refuses them).
- **Modern first-order variants.** AdamW (decoupled weight decay), RAdam (warmup- free), or Lion/ Sophia are drop-in alternatives to Adam. The benchmark harness ([experiments/benchmarks/bench_training.py](#), in `experiments/benchmarks/`) exists to evaluate exactly such swaps on time-to-target.
- **Per-node optimizer selection.** The loss decomposes, and the optimizer already holds one group per node. Therefore different nodes can in principle use different optimizers (for example, L-BFGS for the `ls` nodes and Adam for an MLP node in a mixed model). This is a direction for mixed-flexibility DAGs.
- **Warm-start handoffs.** `fit_classical` $\rightarrow$ `fit` already works (the classical MLE as a fast, principled initialization). The reverse (Adam to escape, L-BFGS to polish) is a natural pattern to formalize.

These are *directions*, not commitments. The autoresearch loop ([docs/research/MISSION_autoresearch.md](#)) is a good venue to test the speed-oriented ones empirically before any adoption.

Notation

This page defines the notation. Every guide, docstring and notebook in this repository follows it.

Symbols

Random variables are capital Latin letters, for example Y . Their realizations — observed or sampled values — are lowercase Latin letters, for example y . The cumulative distribution function of Y is F_Y and the density is f_Y . A hat marks an estimate: $\hat{F}$ is the fitted distribution, F the true and often unknown one.

Sets and vectors are bold, for example $\mathbf{X} = [X_1, \dots, X_J]$.

A single optimized parameter is a lowercase Greek letter, for example ϑ or β . A vector of parameters is bold lowercase, for example $\boldsymbol{\vartheta} = (\vartheta_1, \dots, \vartheta_M)^\top$. A tuple that collects several parameter groups — for example all weight matrices of a network — is bold capital, for example $\boldsymbol{\Theta}$.

A function that a parameter vector defines carries it as a subscript: $h_{\boldsymbol{\vartheta}}$ is the monotone transformation with coefficients $\boldsymbol{\vartheta}$.

The model's symbols

Symbol	Meaning	In code
X_j, Y	the variables of the DAG	node names in the spec
$\text{pa}(X_j)$	the causal parents of X_j	<code>node_parents</code>
U_j	the standard-logistic latent of node j	<code>u = flow.abduct(df)</code>
$h_{\boldsymbol{\vartheta}}$	the monotone transformation of a node	the I term; theta in docstrings
ϑ_k	the ordinal cutpoints, elements of $\boldsymbol{\vartheta}$	<code>ordinal_cutpoints</code>
β	a linear-shift coefficient; e^β is an odds ratio	LS; <code>flow.ls_coefficients()</code>
g_j	a complex shift, an MLP of one term's parents	CS
β_0	the constant treatment effect of a VC term	<code>beta0</code>
$b_{\boldsymbol{\Theta}}$	the penalized effect-modification network	the VC net; <code>units=</code> sets its layers
λ	the L2 weight on $b_{\boldsymbol{\Theta}}$	<code>penalty=</code>
σ	the logistic function	<code>torch.sigmoid</code>

Every node's model is one additive formula on the latent scale:

$$u = h(x \mid \text{pa}(x)) = h_{\boldsymbol{\vartheta}(\cdot)}(x) + \sum_j \beta_j x_j + \sum_k g_k(x_k) + (\beta_0 + b_{\boldsymbol{\Theta}}(x_{\text{mod}})) x_t$$

For a continuous node the shifts are added, as written. For an ordinal node the shift is subtracted inside the sigmoid: $P(Y \leq k \mid \text{pa}) = \sigma(\vartheta_k - \text{shift})$.

Plain-text fallback

Docstrings and terminal output cannot render Greek subscripts. There, `theta` stands for ϑ , `h_theta` for h_{ϑ} , `b_theta` for b_{ϑ} and `beta0` for β_0 .

Per-observation scores & the effect-modifier scan

`flow.scores(df, node)` returns the score $\psi_i = \partial \ell_i / \partial \theta$ of each observation for the node's interpretable shift coefficients. These coefficients are every LS weight and every VC term's `beta0`. An LS weight gives one column per continuous parent and one column per one-hot level of an ordinal parent. The `beta0` column is named after the treatment.

The computation is **analytic and exact**, with no autograd. Shifts enter the latent additively. Thus $\partial \ell_i / \partial \beta = (\partial \ell_i / \partial s_i) \cdot x_i$, with $\partial \ell_i / \partial s_i$ in closed form (`src/tramdag/scores.py`). The function is a pure read-out. It does not touch the fitting or sampling code paths. At a fitted MLE, the sum of each column is ≈ 0 . Tests pin this behavior and include a float64 finite-difference check.

Worked example: where does $\beta(x)$ need to bend?

The score is the cheapest effect-modifier detector available. It uses the structural-change logic of model-based recursive partitioning (Zeileis & Hornik; Dandl et al. 2024). Before you fit an expensive model, fit the **cheap all-LS model**. This fit takes seconds and is deterministic. Then scan the treatment-coefficient scores:

```
flow = td.CausalFlowDAG(all_ls_spec, seed=0)
flow.fit_classical(df) # seconds, exact MLE

scan = flow.effect_modifier_scan(df, node="Y", t="T")
#      stat  p_value crit_5pct  flag
# X2    4.21   <0.001    1.3581  True   <- true modifier
# X3    3.05   <0.001    1.3581  True   <- true modifier
# X1    0.74    0.64    1.3581 False   <- prognostic only
```

For each candidate covariate, the scan orders the scores by that covariate and forms the scaled cumulative sum $B_j = \sum_{i \leq j} \psi_i / (\text{sd}(\psi) \cdot \sqrt{n})$. Under a stable coefficient, B is a Brownian bridge. Thus $\sup |B|$ has the Kolmogorov distribution (5% critical value 1.358). A coefficient that truly varies with the covariate makes the ordered scores drift. The flagged covariates are the measured shortlist of VC modifiers (`docs/varying-coefficients.md`):

```
spec["Y"] = td.ContinuousNode(
  td.CS("X1", "X2", "X3") + td.VC("X2", "X3", t="T")
) # scan-informed
```

Read it as screening, not as a test of effect modification. The statistic measures how unstable the *cheap* model is along a covariate, and instability has two sources: a coefficient that genuinely varies, and a **prognostic part the cheap model gets wrong**. In the example above `X1` is unflagged because its prognostic effect is linear; make it quadratic and `X1` flags as strongly as the true modifiers ($0.56 \rightarrow 4.14$ on the DGP of [notebooks/varying-coefficients.py](#), which demonstrates both cases side by side). A flag therefore means "look here", and what you find may be a modifier or a misspecification — both worth knowing.

Notes: For a binary ordinal LS treatment, `t` resolves to the identified level-1-vs-0 contrast. For a VC treatment, `t` resolves to `beta0`. Candidates default to every column of `df` except the node and `t`. Modifiers do not need to be parents. For few-level (heavily tied) candidates, the ordering is only partial. Then read the scan as a ranking diagnostic, not as an exact-size test. You can also use the scores later for influence-function analyses and robust standard errors.

How fast can a CausalFlowDAG train?

This document benchmarks learning-rate schedules, per-node freezing, batch sizes, devices, and LBFGS. The benchmark ran in June 2026 on an Apple-silicon Mac mini with torch 2.12 (CPU unless noted). To reproduce it, run `experiments/benchmarks/bench_training.py` (`cd experiments && uv run python -m benchmarks.bench_training`, or `--quick` for one seed on cpu); the full grid takes ≈ 35 min. For a quick **cross-machine** comparison, use the self-contained `experiments/benchmarks/perf_machine.py`. It runs fixed 200-epoch workloads on all available devices and writes a machine fingerprint to JSON, and it needs nothing but `pip install tramdag`. The raw CSV is a local run artifact and is not committed.

The options, and how to use them

Schedules (`fit(..., schedule=...)`) control the learning rate over training:

value	behavior
None (default)	constant lr — the classic behavior, exactly as before this PR
"plateau"	per-node : a node whose own validation NLL hasn't improved by <code>min_delta</code> (default $1e-4$) for <code>plateau_patience</code> epochs (default 30) gets its $lr \times 0.3$, floored at $1e-3 \times$ the initial lr. Each node decays independently — valid because the per-node losses have independent gradients.

"onecycle" and "cosine" were part of this benchmark and lost to "plateau" on every workload; 0.4.0 removed both. The result tables below keep their measured rows as the record of that decision.

Early stopping / freezing (`fit(..., freeze_patience=N)`): if the validation NLL of a node does not improve for N epochs, the fit *freezes* that node. A frozen node leaves the loss and the backward pass, which saves real compute. Its weights stay fixed from then on. When all nodes are frozen, the fit returns early. The fit records the freeze epochs in `flow.history["frozen"]`. Under `schedule="plateau"`, freezing also waits until the lr of the node is decayed $\geq 100\times$. This delay prevents a freeze while a smaller step size can still make progress. Freezing state is per-`fit()`-call: a second `fit` call trains all nodes again.

Switching it all off: for an exact comparison with classical methods (`statsmodels`, R `polr`/`tram`), omit both arguments. The benchmark changed no defaults, so

```
flow.fit(train_df, epochs=4000, learning_rate=1e-2, batch_size=512) # constant lr
flow.fit(train_df, epochs=2000, learning_rate=1e-3, batch_size=512) # 2nd phase
```

is still the exact-MLE path that `experiments/misc/validate_ls.py` uses. Independent of all this, `restore_best=False` remains the default (see CHANGELOG). The guard test `tests/test_fit_schedules.py::test_plateau_freeze_preserves_exact_mle` also shows that even *with* plateau+freezing the all-`ls` fit lands on the classical MLE within the usual tolerances.

LBFGS is *not* a `fit()` option — it ships as its own method. `fit_classical` runs float64 full-batch L-BFGS with a strong-Wolfe line search and supersedes the hand-rolled recipe this report benchmarked: it is deterministic, lands on the exact MLE, matches `statsmodels`/R `polr`, and refuses non-`ls` specs (minibatch noise also regularizes the MLPs, so flexible models belong in `fit`). The float32 single-precision variant measured here was fast (< 2 s) but not robust across seeds (Findings #2) — `fit_classical`'s float64 upcast is what fixed that.

```
report = flow.fit_classical(train_df) # exact classical MLE, seconds
```

Method: time-to-target, not loss-go-down

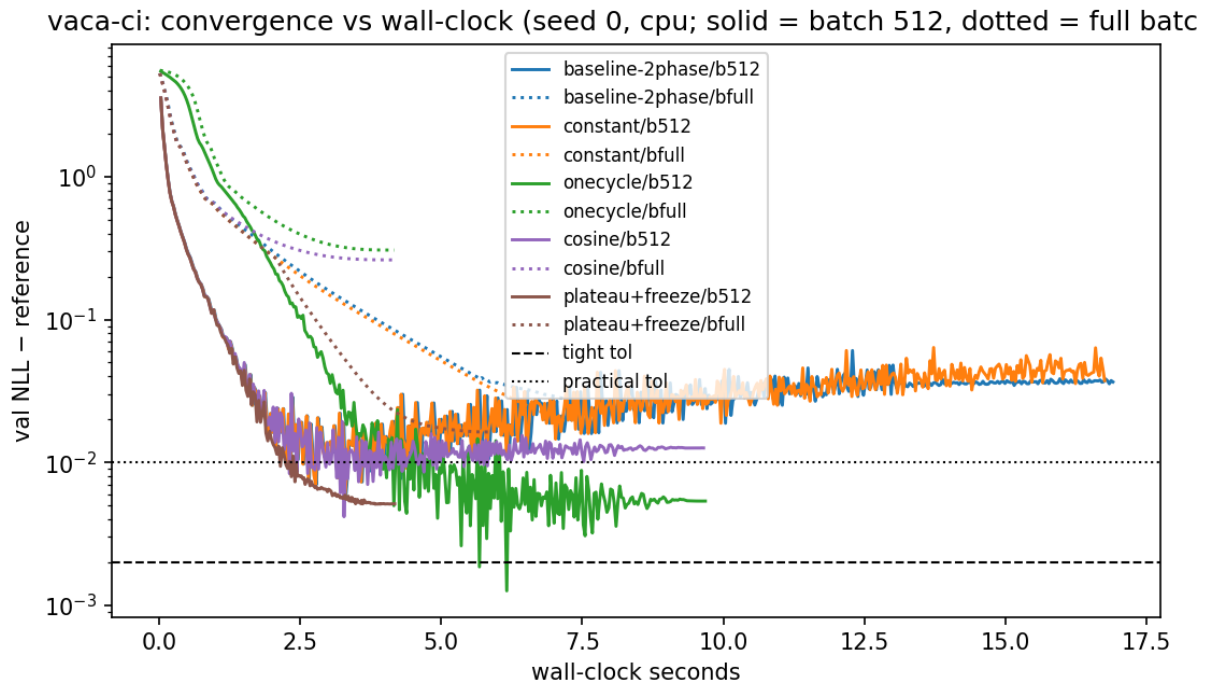
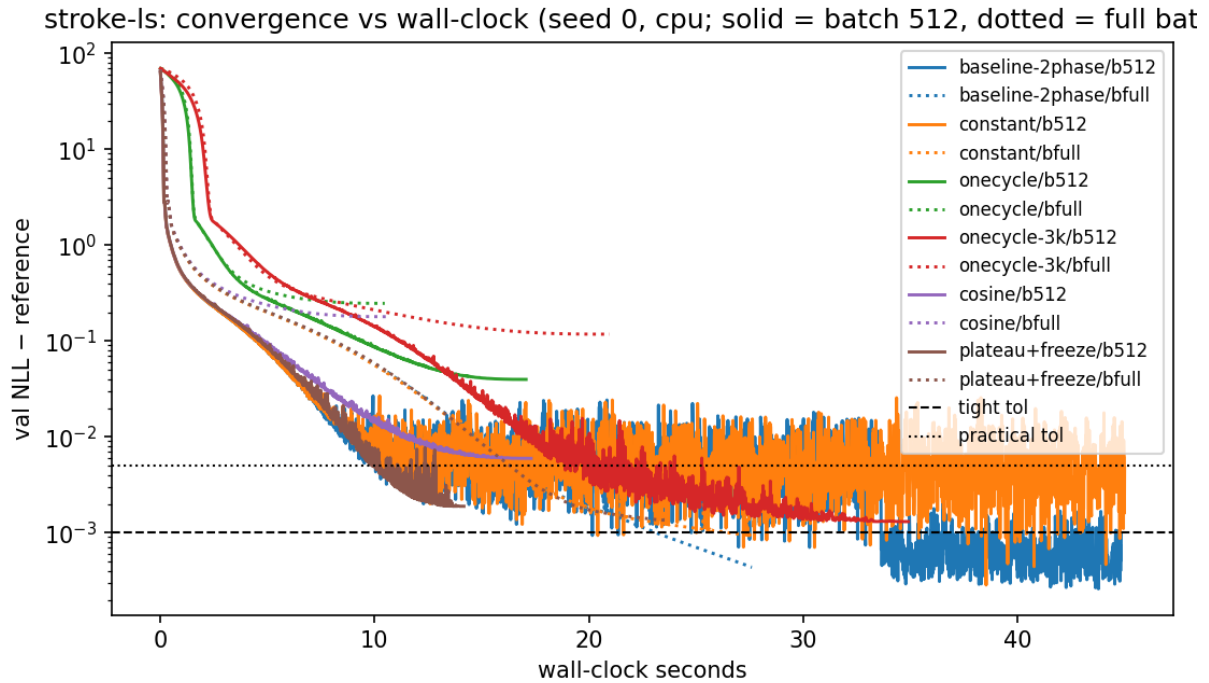
Each config runs once. `fit()` records per-epoch validation NLL *and* wall-clock time. From these records, we measure the seconds until the fit is within a fixed gap of a cached long-run reference (3 torch seeds, medians):

workload	model / data	reference NLL	tight tol	practical tol
stroke-ls	all-ls 5-node DAG, frozen experiments/misc/data/magic-mrclean/ls (n=1275, full-data MLE)	10.3042 (train)	+1e-3	+5e-3
vaca-ci	all-ci flow, frozen experiments/paper/data/vaca (n=5000, 90/10 split)	4.9632 (val)	+2e-3	+1e-2

Tight $\approx$ exact-MLE equivalence (statsmodels/R-polr match). *Practical* $\approx$ coefficient-equivalent: a fit with gap $\approx 3e-3$ already matches the R reference coefficients within the tolerances of [experiments/misc/validate_ls.py](#). The same exact-MLE-under-plateau-and-freezing property is pinned on an inline DGP by `tests/test_fit_schedules.py::test_plateau_freeze_preserves_exact_mle`.

These numbers were measured before the experiment code moved into `experiments/benchmarks/`; the workloads are unchanged (same frozen data, same specs), so the timings still stand. Re-running the benchmark reproduces the machine-independent part exactly: the stroke-ls reference NLL 10.3042.

Results



The table shows median seconds to target (batch 512, cpu). A "—" entry means that the fit never reached the target within the budget.

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
baseline two-phase (old default)	9.0	21.4	2.1	2.8 ¹	no (runs 40 s / 15 s)

config	stroke-ls practical	stroke-ls tight	vaca-ci practical	vaca-ci tight	self-stops
constant 1e-2	9.1	21.5 ³	2.2	2.8 ¹	no
onecycle (1500 / 300 ep) ²	—	—	3.5	4.5	no
onecycle (3000 ep) ²	16.8	— (gap 1–2e-3)			no
cosine ²	—	—	2.2	3.5 ¹	no
plateau + freeze	8.9	— (gap 2e-3)	2.0	2.9	yes — 13 s / 4 s total
LBFGS (full-batch)	1.6 (2/3 seeds)	— (gap 4–8e-3)	n/a	n/a	yes

¹ transient: the val-NLL curve dips through the target and then drifts away. Stroke needs the 1e-3 phase to stay. Vaca shows mild overfitting. Final gap for the vaca baseline is 0.037. The old 520-epoch budget **underfits** vaca by ~0.03 nats. Plateau+freeze stays at its target. ² onecycle and cosine were removed from `fit()` in 0.4 — they lost to plateau on every workload here — so these three rows cannot be re-measured. ³ constant lr at batch 512 stalls at gap 3–7e-3. Only the lr-decay phase closes the last decade. This is why the two-phase recipe existed.

Findings

- Per-node plateau decay + freezing is the best default-style trainer.** It has the same time-to-accuracy as the hand-tuned two-phase schedule, but it needs **no budget tuning**. It decays the lr of each node off its own validation curve. It freezes converged nodes, which is a real FLOP saving because the per-node NLLs have independent gradients. And it **stops itself**: 13 s total vs 40 s for the baseline on stroke-ls, 4 s vs 15 s on vaca-ci, at equal or better final NLL.
- LBFGS is spectacular but not robust.** Full-batch LBFGS reaches coefficient-level accuracy on the classical all-1s model in **< 2 s** (vs 9 s for Adam) on 2/3 seeds. The third seed stalls at gap 8e-3. An Adam warm start made it *worse* (different basin) on every seed. Use it as a fast first shot with the plateau trainer as fallback, not as the default.
- OneCycle is a "spend exactly this budget" scheduler.** Accuracy arrives only at the end of its anneal. At 1500 epochs it misses everything. At 3000 epochs it lands gap 1–2e-3. But you must know the right budget in advance, and this requirement is the problem that we try to remove.
- Full-batch loses on time-to-target** despite ~1.6× higher epoch throughput. It makes too few optimizer steps per second of compute at these n. Batch 512 is a good default. Very large batches (16k) only improved raw throughput at n=50k.
- MPS (Apple GPU) is 3–4× slower than the M-series CPU** at these model sizes (verified correct: identical reconstruction). Kernel-launch overhead dominates sub-millisecond ops. Stay on CPU locally. CUDA on Colab-class GPUs is a different regime (see the GPU-vs-CPU race in the demo notebook).
- The old defaults waste or under-spend.** Stroke: 4000 epochs budgeted, converged work done after ~1500 (freezing recovers the difference automatically). Vaca: 520 epochs budgeted, ~0.03 nats short of converged. Fixed budgets are wrong in both directions. Adaptive stopping fixes both.

Recommendation

For everyday fits:


```
flow.fit(
    train,
    val,
    epochs=4000,
    learning_rate=1e-2,
    batch_size=512,
    schedule="plateau",
    plateau_patience=30,
    freeze_patience=120,
)
```

The generous `epochs` value is only a ceiling. The fit stops itself. For exact classical comparisons where the last 1e-3 matters, append a short constant-lr polish phase (`epochs=500, learning_rate=1e-3`) after the plateau fit. Or run the old two-phase recipe.

The benchmark itself changed no defaults — that is its own reviewed decision (see the `restore_best` episode in `CHANGELOG.md`). Two of its findings have since been adopted: `plateau_patience` defaults to the 30 recommended here, and `epochs` has no default at all, because finding 6 is precisely that a fixed budget cannot be right for every workload. The experiment scripts still run the two-phase constant-lr recipe, which each states in its own YAML.

Varying-coefficient treatment effects: `vc(*modifiers, t=, penalty=)`

A `vc` term gives a node a **treatment-effect head with its own bias–variance budget** (issue #28). The term contributes

```
beta(modifiers) * x_on      with      beta(x) = beta0 + b_theta(x)
```

to the node's shift. Here `b_theta` is a deliberately small (one 16-unit hidden layer), **penalized** network. The point is not expressiveness. The point is that the flow estimates the effect function *with care*, not as a by-product:

```
import tramdag as td

spec = {
    "X1": td.ContinuousNode(),
    "X2": td.ContinuousNode(),
    "X3": td.ContinuousNode(),
    "T": td.OrdinalNode(2, td.LS("X1") + td.LS("X2")),
    "Y": td.ContinuousNode(
        td.CS("X1", "X2", "X3") # prognostic part g(x): as flexible as you like
        + td.VC("X2", "X3", t="T", penalty=1.0) # effect beta(X2, X3) * T
    ),
}
flow = td.CausalFlowDAG(spec, seed=0).fit(
    train, val, epochs=500, learning_rate=1e-2, batch_size=512, restore_best=True
)

beta = flow.varying_coef("Y", df_new) # (n,) array beta(x) – deterministic, y-free
beta0 = float(flow.nodes["Y"].shifts["T"].beta0) # interpretable main effect
```

`x2` and `x3` appear **twice**: as prognostic parents through `cs` and as effect modifiers through `vc`. This pattern is intended. Only the treatment node named by `t=` owns its edge, and a second term that declares it raises an error. Modifiers can repeat.

Why not `cs("T", "X2", "X3")` ? (anti-pattern)

For a binary treatment, the multi-parent `cs` is equivalent in *expressiveness*. Any shift decomposes exactly as $s(x, t) = s(x, 0) + [s(x, 1) - s(x, 0)] \cdot t$. But the `cs` form has **no effect-specific regularization**. The likelihood rewards a good average fit of $s(x, t)$, and nothing rewards a smooth difference between the arms. The read-out is thus the difference of two jointly fitted, unregularized networks, which amplifies noise.

On the heterogeneous-effect validation DGP (see below), the `cs` reduced form reaches $\text{corr} \approx 0.5$ against the true effect function (tramdag-simu#18 / PR #21). This holds **even when the model is exactly in-class**. The `vc` term reaches $\text{corr} \approx 0.99$ on the same protocol (`tests/test_vc_term.py`, acceptance bar 0.9). Causal forests and R-learners work not because they *target* the effect, but because they **regularize** it (Nie & Wager 2021, Athey–Tibshirani–Wager 2019). `vc` brings that ingredient into the TRAM framework.

Semantics

- **Scale:** $\text{beta}(x)$ lives on the node's latent (log-odds) scale. The flow adds it for a continuous node ($u = h(y) + \dots$) and subtracts it from the cutpoints for an ordinal node, exactly like an `LS` weight. With no modifiers, `vc(t="T")` is `LS("T")` (identical model, testably bit-exact). Thus `vc` versus `LS` is a nested question.

- **Penalty:** the fitting objective is the penalized likelihood $\sum_i \text{NLL}_i + \text{penalty} \cdot \|b_{\text{theta}} \text{ weights}\|^2$ on the total-NLL scale. This is a fixed Gaussian prior whose shrinkage vanishes as n grows. The penalty never applies to β_0 . $\text{penalty} \rightarrow \infty$ shrinks b_{theta} to the zero function and recovers the classical LS fit. The default is $\text{penalty}=1.0$. If the modifiers are many or n is small, raise the penalty.
- **Identification / centering:** a constant moves freely between β_0 and b_{theta} . The head's output layer is zero-initialized ($\beta(x) = \beta_0$ at step 0). After `fit`, the flow re-centers the head to mean zero over the training data, which preserves the function. Thus β_0 is the main effect in the training population, the `Colr` reading when β is constant.
- **Warm start:** `fit(vc_warm_start=True)` (the default) initializes β_0 from the classical all-`ls` solution of the node's conditional, once per term. That solution comes from a deterministic L-BFGS on a throwaway proxy. Thus training starts at the classical answer and only learns deviations.
- **Treatments:** the treatment `x_on` can be continuous or binary (2-level) ordinal. The term is linear in `x_on`. A binary ordinal enters as its 0/1 level, so β is the identified level-1-vs-0 contrast. Multi-level ordinal treatments are a planned follow-up.
- **Read-out:** `flow.varying_coef(node, data, t=...)` evaluates $\beta_0 + b_{\text{theta}}(\text{modifiers})$ in closed form. The result is deterministic and y -free, and it needs no abduction. For a binary treatment, it equals the abduct-difference $u(x, t=1, y) - u(x, t=0, y)$ identically (pinned by a test).

Propensity-centered VC: `center=True` (R-learner orthogonalization)

```
td.VC("X2", "X3", t="T", penalty=1.0, center=True, center_folds=5)
# contributes  $\beta(x) * (t - \hat{e}(x))$  to the shift
```

This is Robinson/R-learner centering inside the likelihood. Dandl et al. (2024) found treatment-centering to be *the* decisive ingredient for effect estimation under confounding in model-based forests. That finding reproduces here. The test DGP is strongly confounded, and the model deliberately under-specifies its prognostic part (true $g(x)$ quadratic, model linear). On that DGP, the uncentered β absorbs the confounded misfit. The mean $|\beta - \tau|$ is ≈ 1.1 – 1.2 , which effectively destroys the effect estimate. The centered β stays near truth (≈ 0.1 – 0.3), a **5–10× bias reduction** (`tests/test_vc_centered.py`). If the prognostic part is correctly specified, centering changes little. Centering is insurance against the misspecification that you do not know you have.

The naive implementations are wrong. Thus this is a **two-stage frozen** design:

- **Training** uses **out-of-fold** $\hat{e}$: the fit does `center_folds` refits of the treatment node *only*, and each refit predicts the fold that it never saw. This is the DML cross-fitting requirement. In-sample $\hat{e}$ reintroduces the own-observation bias and can be *worse* than no centering. The OOF values enter the outcome loss as frozen data. Thus **no gradient reaches the treatment node** from the outcome node, and the per-node factorization stays intact (pinned by a gradient-isolation test). The flow exposes the fold bookkeeping in `flow.vc_center_info`, pinned by tests.
- **Inference** (`log_prob` / `sample` / `abduct` / `pmf` / `scores`) recomputes $\hat{e}$ from the flow's **own fitted treatment node**, the full-data fit (the standard DML train/predict split). The computation is detached and uses the current parent values. Under `do(T=t)`, the flow re-derives the regressor as $t - \hat{e}(x)$. The flow caches nothing.
- **Interpretation:** with centering, β_0 is the effect at the treatment margin (the observed propensities). `varying_coef` is unchanged, because centering moves the regressor, not β . The LS-nesting reading applies to the uncentered term only. Centering requires a binary ordinal treatment. Continuous-treatment centering with $E[T|x]$ is a follow-up. `center=False` (the default) is bit-identical to the uncentered term.

Validation

The `vc_hetero` DGP in [tests/conftest.py](#) is a logistic-shift SCM with known `beta_true(x) = -1 + 0.8·X2 - 0.6·X3`. It has a nonlinear prognostic part and confounded assignment (X2 is confounder *and* modifier), which is the configuration where the `cs` reduced form fails hardest. Acceptance (`tests/test_vc_term.py`) requires three results: recovery `corr ≥ 0.9` at `n = 5000` (measured ≈ 0.99 , min over 3 seeds 0.986), a fitted `beta0` that matches `fit_classical` under a large penalty, and the read-out identities. The centering claims are measured against the `confounded` DGP in the same file. Both former follow-ups have since shipped: propensity centering (#30) is `center=True`, documented above, and the per-observation scores for effect-modifier scans (#29) are `flow.scores` and `flow.effect_modifier_scan` — see [scores.md](#).